

Predicting When Training-Free Sparse Prefill Pays on Real Host-GPU Pairings: Research Gaps, Contributions, Methodology and Closest Prior Work

Mehul Dahiya · Research proposal brief · 4 October 2026

Research question. Can the end-to-end speedup and the accuracy of training-free block-sparse prefill attention be predicted for host-GPU pairings on which the workload has not been run, using component measurements alone, and where does the prediction fail?

Background. Block-sparse prefill skips attention blocks judged unimportant by an estimator. Papers usually report a kernel speedup on one machine. Our earlier measurements show that this kernel speedup often does not survive end to end: on an A100 the sparse kernel was 1.28x to 1.96x faster than FlashAttention, yet end-to-end prefill became slower (0.63x to 0.75x) because the host CPU built the masks on the critical path. The same configuration was faster on an L4, whose slower GPU hid the host work. Whether sparse prefill pays therefore depends on the host, the GPU and how their work overlaps, not on the kernel alone. This matters more on each new GPU: kernel-launch latency is almost flat across generations (about 6.1 microseconds on a V100, 6.0 on an A100 and 4.15 on an H100, IEEE LANMAN 2026) while attention kernels keep getting faster, and recent work reports sparse methods running slower than dense under chunked prefill (CompactAttention).

1. Research gaps

Gap	What is missing	Closest work and what it leaves open
G1	End-to-end latency on unseen CPU-GPU pairs can already be predicted with a learned model from host, transfer and device micro-benchmarks (CTFusion). It has not been tested whether an explicit host-GPU critical-path model predicts data-dependent sparse prefill on unseen pairings better than such a learned predictor, especially when only a few hardware systems are available for training.	CTFusion (IEEE FLTA 2025) predicts object-detection latency on unseen CPU-GPU pairs (17.6% error) with gradient-boosted trees trained on 20 systems; its host side is input preprocessing and the host-GPU interaction is learned, not modelled. nn-Meter, HELP and CloserToMe predict model latency on unseen devices, treating each device as one unit. KernelSight-LM assumes its host costs do not depend on the GPU but does not test this. None models data-dependent work and synchronisation inside a forward pass.
G2	No lightweight model represents data-dependent mask generation and its GPU-to-CPU synchronisations inside a prefill step, which decide how much host time is exposed.	TaxBreak's HDBI is a ratio of sums and TKLQT an aggregate; KernelSight-LM uses one maximum per step plus a serial tail; LLMservingSim 2.0 models synchronisation between devices, using profiles of each model and device. None follows where data-dependent host work and synchronisations fall within a prefill step, and TaxBreak notes its replay method can be imperfect for synchronisation-heavy kernels.

G3	It is untested whether attention recall (the share of the true attention mass the kept blocks capture) is a sufficient proxy for task accuracy across different estimators.	VSPrefill shows accuracy rising with recall on one dataset, and compares recall, not accuracy at the same recall, across selection methods. Less Is More treats recall as necessary but not sufficient in long decoding. The Sparse Frontier names the question as open. An oracle-guided prefill report (entry 26) lists a related check, how much of the oracle's choice a learned selector recovers, as future work. Whether two different estimators at the same recall give the same accuracy has not been tested.
-----------	---	---

Where the novelty lies. G2 and G3 are the main gaps. G1 is narrower: the problem of predicting latency on unseen pairs is already addressed by CTFusion, so G1 is kept as the validation question, whether an explicit model helps when only a few hardware systems are available for training.

2. Contributions

Each contribution addresses one or more gaps. All predictions are written down before the measurements that test them, and failures will be reported as failures.

- 1. A component-composed critical-path model for sparse prefill (G1, G2).** The model splits every operation into a host cost, a device cost and a link cost, and composes them in execution order with the synchronisation points of each estimator. It is compared against three alternatives built from the same measurements: an overlap-free sum, the step form of KernelSight-LM (an overlap floor plus a serial tail for synchronisations), and a learned predictor in the style of CTFusion (gradient-boosted trees on micro-benchmark features). If the learned predictor does better on unseen pairings, that is reported.
- 2. Validation on physically distinct hardware without running the workload first (G1).** Seven real host-GPU pairings across two cloud providers, with five GPUs and four host CPU platforms. For each pairing the prediction is computed from component measurements, hashed and saved before the end-to-end workload is run. One pairing (H100 on an AMD host) is predicted entirely from components measured on other pairings, and one host platform (Sapphire Rapids) is held out as a forecast.
- 3. A separability test for host and device costs (G1).** A direct test of whether host costs measured with one GPU carry over to another GPU on the same host platform, and whether device costs carry over between host platforms for the same GPU. This decides when component costs can be reused across hardware. The same measurements test how the crossover point moves as the GPU gets faster relative to the host.
- 4. A test of recall as an estimator-independent proxy for accuracy (G3).** Several estimators are compared at matched recall, matched layer by layer, at block sizes 32, 64 and 128, with results also reported for shallow, middle and deep layers. If recall is sufficient, cheap recall measurement can replace expensive accuracy runs; if not, the result locates where dropped attention mass matters.

What is already in place

Item	Status
Earlier study on L4, A100 and H100	Done. Host-side mask building explains why kernel speedup does not survive end to end. Two of three predictions registered before the H100 run passed.
Accuracy pilots on the L4 (Qwen2.5-1.5B, 16K and 32K tokens)	Done. With oracle masks, accuracy is non-inferior to dense only at 50% density, and only in two of the tested cells. The mean-pool estimator and XAttention with a single threshold were not non-inferior in any cell; XAttention with calibrated per-head thresholds is built and not yet run.

Measurement harness	Built and tested: one shared block selector and kernel for all estimators, commit-stamped rows, exact statistical analysis.
Component profilers, cost model, AWS tooling	To be built.

3. Methodology

3.1 Hardware pairings

Google Cloud fixes the host CPU by machine type, so the design crosses two providers. Three GPUs share one AMD host platform, and the L4 and the H100 each appear on two different hosts.

Pairing	Host CPU	GPU	Provider	Purpose
P1	Intel Cascade Lake	L4	GCP G2	Anchor pairing.
P2	Intel Cascade Lake	A100	GCP A2	Same host platform as P1, different GPU.
P3	AMD EPYC (3rd gen)	L4	AWS G6	Same GPU as P1, different host platform.
P5	AMD EPYC (3rd gen)	L40S	AWS G6e	Second GPU on the AMD host; faster GPU, same host.
P4	Intel Sapphire Rapids	H100	GCP A3	Host platform used nowhere else: forecast.
P6	AMD EPYC (3rd gen)	H100	AWS p5.4xlarge	Same GPU as P4 on another host; predicted only from other pairings' components. Run last.
P7 (optional)	AMD EPYC (2nd gen)	A10G	AWS G5	Another host generation, if time allows.

Run order: P1, P2, P3, P5, P4, P6, then P7, so that every reused component exists before the pairing that uses it.

3.2 Measurement protocol for each pairing

- Environment.** The same container image on both providers; driver, CPU model, PCIe link, NUMA layout and clock state recorded on every row; threads pinned to the CPU node closest to the GPU; CPU frequency sampled throughout.
- Link costs.** Short microbenchmarks: host-device bandwidth from pinned and from pageable memory, 1-byte copy, synchronisation round trip, and the depth of the GPU launch queue.
- Host costs.** Each estimator's operation sequence with device work replaced by empty kernels, plus host-side computation (such as CPU mask building) run directly at the target size.
- Device costs.** Each operation replayed on its own on the GPU under Nsight Systems, with the masks recorded for each prompt. The same method is used for every estimator. Because TaxBreak warns that replay can be imperfect for synchronisation-heavy kernels, replayed kernel names and durations are first compared with the same kernels in a traced run on P1.
- Prediction saved.** Predicted end-to-end time computed, hashed and synced to storage before step 6.
- Ground truth.** Untraced end-to-end prefill for every estimator, length and batch size (1 and 4), with a dense control in every session.

3.3 Cost model

For operation i , h_i is its host cost, d_i its GPU execution time and l_i its link cost (host-device copy time, zero for a kernel); s is the measured cost of one synchronisation round trip. Only device costs

are carried between hosts; link costs are always measured on the target pairing. H_i is when the host has issued the operation and D_i is when the GPU finishes it:

$$H_i = H_{i-1} + h_i$$

if the launch queue is full (Q pending): $H_i = \max(H_i, D_{i-Q})$

at a synchronisation point: $H_i = \max(H_i, D_{i-1}) + s$

$$D_i = \max(H_i, D_{i-1}) + l_i + d_i$$

predicted end-to-end time = D_N

The model covers one GPU stream, where copies and kernels run one after another, so it does not assume they overlap. Every trace is checked for copies running at the same time as kernels; where that happens, the slowdown is measured on that pairing and added as data, never fitted. The three baselines use the same measurements: sum = $\Sigma h + \Sigma l + \Sigma d$ (no overlap), KernelSight-style = $\max(\Sigma h, \Sigma l + \Sigma d)$ per layer plus every synchronisation cost added at the end, and a CTFusion-style learned predictor trained with one pairing left out at a time. The KernelSight-style form uses the same synchronisation costs as our model but ignores where they fall, so beating it tests G2; beating the learned predictor tests G1. Stated in advance: on dense prefill, which has no synchronisation inside the step, our model and the KernelSight-style form should agree; an advantage is expected only for estimators with synchronisations. HDBI is reported alongside as a diagnostic.

Ablation. Each estimator is also run with its masks computed in advance and preloaded on the GPU. The difference from the live run separates the estimator's own cost and synchronisations from the kernel.

3.4 Hypotheses

#	Hypothesis	How it is tested
H0	Separability: host costs carry over between GPUs on the same host platform; device costs carry over between host platforms for the same GPU. (KernelSight-LM assumes the host half of this without testing it.)	Host side: P1 with P2, and P3 with P5 and P6. Device side: P1 with P3 (L4) and P4 with P6 (H100). Per operation, within a tolerance fixed in advance.
H1	Composition: on every pairing the recursion predicts end-to-end time within tolerance, and on unseen pairings better than all three baselines.	Relative error per pairing; sparse-versus-dense direction correct.
H2	Transfer: predictions stay within tolerance when components come from other pairings, including P6, where both host and device costs come from other pairings.	Same statistic as H1; tested only if H0 holds.
H3	Forecast: on P4 the context length at which each estimator beats dense falls inside the band predicted before P4 is run.	Predicted band against measured crossover with bootstrap intervals.
H4	Recall sufficiency: at matched recall in every layer, accuracy does not depend on the estimator, at block sizes 32, 64 and 128.	Paired comparison against an equivalence bound fixed in advance; also reported separately for retrieval-like and local heads, and for the first, middle and last thirds of layers.
H5	Direction (secondary): on a fixed host, a faster GPU moves the crossover to longer contexts for estimators with host-side steps, less so for estimators that run fully on the GPU; on a fixed GPU, the cheaper host gives the earlier crossover.	Directions predicted from measured cost ratios before data (L4, L40S and H100 on the AMD host; the L4 and the H100 each on two hosts).

Extension. On P5, prefill is also run in chunks (as serving systems do) for two estimators, and the extra selection cost per chunk is predicted before the run.

3.5 Estimators and accuracy evaluation

- **Estimators:** oracle (exact attention mass; a reference, not a guaranteed quality ceiling, since the mask that keeps the most attention need not be best for the task), sink-plus-local window (zero-cost floor), mean-pool (MInference style), vertical-slash style, XAttention with per-head calibrated thresholds, and DuoAttention-style head masks where the authors released patterns. Dense attention uses the fastest correct kernel on each GPU.
- **Recall:** at each layer, the share of that layer's dense attention mass, computed from its actual inputs in the same forward pass, that the kept blocks capture. Recall against a separate dense pass is reported as a second measure.
- **Models and tasks:** Qwen2.5-1.5B (primary), Qwen2.5-7B (recall and cost), Llama-3.1-8B (held out of all fitting); RULER tasks at 16K and 32K tokens, with a real-text haystack version planned.
- **Where quality is measured:** once, on the primary GPU, with the dense control repeated on a second GPU.
- **Accuracy test (fixed):** an exact paired non-inferiority test (union-bound Clopper-Pearson), margin 10 percentage points, one-sided alpha 0.025, the same test used in the completed pilots. A cell counts only if dense accuracy is above a floor fixed in advance.

3.6 Controls

Risk	Control
Different CPUs, caches, memory, PCIe	Host and link costs are measured on each real pairing, never simulated.
Boost clocks and virtual-machine noise	Cloud VMs cannot fix, and often cannot read, the CPU clock, and CPU frequency can change CPU-GPU timing in a non-monotonic way (Elango et al., IEEE INFOCOM 2026). So host speed is measured directly: a fixed CPU loop and a burst of empty GPU launches are timed at the start, middle and end of every session. A session is used only if this probe is stable within a bound, and costs are carried between sessions only if their probe results agree within a bound; both bounds are fixed in advance from repeat sessions. Threads pinned, warm-up, interleaved runs.
CPU mask building competing with copies for host memory	Copy bandwidth also measured while the CPU runs a mask-sized load; copies that overlap host-side estimator work use the loaded figure.
Differences between cloud providers	Same container image; before H2 is read, the same dense workload is run on P1 and P3 to measure provider-level variance, and H0 tolerances account for it.
Fitting the model to its own results	Predictions hashed and saved before the workload they predict is run.
Profiler overhead	Ground truth is always untraced; traced-versus-untraced difference reported.
CPU-GPU placement and host memory type	Threads pinned to the GPU-local CPU node; pinned and pageable copies measured separately; any unified-memory page fault flags the row.
GPU launch queue filling up	Queue depth measured per pairing and included in the model; every prediction is also recomputed with the depth at one quarter to four times the measured value, and without the queue term, to show how much it matters.

Limits stated in advance. Seven pairings is still a small number, so results are reported per pairing and the claim covers only the measured hosts and GPUs. The main claim is predicting time at a given density: in the pilots no deployable estimator matched dense accuracy at any tested density, so profitability for those estimators may turn out to be empty, and that will be reported. Models that need several GPUs (70B parameters and above) are out of scope. Replaying operations one at a time may miss some interactions between neighbouring operations; the error in H1 measures how much this matters.

4. Closest prior work

Ordered from closest to our contributions to more distant, with three further recent preprints at the end. "Preprint" means not yet published in a peer-reviewed venue at the time of writing.

#	Paper	Venue	Year	Relation to this work
1	A Holistic Model for End-to-End Latency Prediction of Deep Learning Tasks on Heterogeneous Systems, CTFusion (Gennai, Ohsita and Shimonishi)	IEEE FLTA (Int. Conf. on Federated Learning Technologies and Applications)	2025	Predicts end-to-end latency on unseen CPU-GPU pairs from host, transfer and device micro-benchmarks with gradient-boosted trees. Closest to G1; our learned baseline follows it. Its workload is object detection with a preprocessing host side, not data-dependent work inside a forward pass.
2	TaxBreak: Unmasking the Hidden Costs of LLM Inference Through Overhead Decomposition (Vellaisamy et al.)	IEEE ISPASS	2026	Splits host overhead per kernel into framework, library and launch costs, using isolated replay, and defines HDBI; two H100/H200 systems. Closest to G2; does not model where synchronisations fall or predict across pairings. Our host-cost method follows it.
3	KernelSight-LM: A Kernel-Level LLM Inference Simulator	arXiv preprint (2606.28565)	2026	Predicts serving latency on an unseen GPU; host constants measured per host and combined with GPU time as an overlap floor plus a serial tail. Closest to G1 and G2; dense attention only, no data-dependent work inside a step, and the host constants are assumed not to depend on the GPU. One of our baselines.
4	APEX: Asynchronous Parallel CPU-GPU Execution for Online LLM Inference on Constrained GPUs (Fan et al.)	IEEE IPDPS (Int. Parallel and Distributed Processing Symposium)	2026	Uses offline profiles to decide when CPU-offloaded decode attention should overlap GPU work, on two machines. Shows that moving a synchronisation point changes exposed time, which supports G2's premise; it schedules on profiled hardware and does not predict time on an unmeasured pairing.
5	nn-Meter: Towards Accurate Latency Prediction of Deep-Learning Model Inference on Diverse Edge Devices (Zhang et al.)	ACM MobiSys	2021	Kernel-level latency prediction on unseen edge devices. Closest learned-prediction precedent for G1; fixed model graphs, host and device as one unit.
6	HELP: Hardware-Adaptive Efficient Latency Prediction for NAS via Meta-Learning (Lee et al.)	NeurIPS	2021	Predicts latency on an unseen device from about 10 measurements. Related to G1.
7	CloserToMe: A Unified Framework for Accurate and Transferable Latency Prediction Across Heterogeneous Devices	AAAI	2026	Predicts latency of small benchmark networks on unseen GPUs, FPGAs and mobile chips from a hardware description and a few probe runs on the target. Related to G1; each device is one unit, with no host-GPU split.

#	Paper	Venue	Year	Relation to this work
8	LLMServingSim 2.0: A Unified Simulator for Heterogeneous and Disaggregated LLM Serving Infrastructure (Cho et al.)	IEEE ISPASS	2026	Serving simulator whose execution graphs include synchronisation between devices, built from profiles of each model and device. Related to G2; models serving systems, not data-dependent host work inside a prefill step, and does not predict an unprofiled pairing.
9	VSPrefill: Vertical-Slash Sparse Attention with Lightweight Indexing for Long-Context Prefilling	arXiv preprint (2603.04460)	2026	Accuracy rises with recall on one dataset; recall, not accuracy, compared across selection methods. Closest to G3.
10	Less Is More: Fast and Accurate Reasoning with Cross-Head Unified Sparse Attention (Yang et al.)	arXiv preprint (2508.07101)	2026	Treats recall as necessary but not sufficient in long reasoning decodes; on one problem, higher-recall settings still fail. Compares recall across methods, not accuracy at the same recall, and studies decoding, not prefill. Related to G3.
11	CompactAttention: Accelerating Chunked Prefill with Block-Union KV Selection	arXiv preprint (2605.16839)	2026	Under chunked prefill on two GPUs, XAttention's end-to-end time is 0.68x to 0.84x of dense speed, because of short query chunks and per-chunk mask search. An external reference for our chunked-prefill extension. Accuracy is compared at settings chosen per method, not at the same recall.
12	Characterizing and Optimizing LLM Inference Workloads on CPU-GPU Coupled Architectures (Vellaisamy et al.)	arXiv preprint (2504.11750)	2025	Three host-GPU systems. TKLQT adds up launch and queueing time per kernel to find where prefill stops being CPU-bound as batch size grows; host single-thread speed dominates at small batches. Host and GPU change together across its systems, and it does not predict time. Its Sapphire Rapids plus H100 launch figure (2.37 microseconds) is a second reference for our P4 check.
13	Vidur: A Large-Scale Simulation Framework for LLM Inference (Agrawal et al.)	MLSys	2024	Profile-based inference simulator, under 9% error, on the profiled GPU. Related to G1.
14	Hardware-Assisted Cross-Generation Prediction of GPUs Under Design	IEEE Transactions on Computer-Aided Design (TCAD)	2019	Predicts next-generation GPU performance. Prediction across hardware, but for GPU design, not host-GPU pairing.
15	The Framework Tax: Disparities Between Inference Efficiency in NLP Research and Deployment (Fernandez et al.)	EMNLP	2023	Shows that operation-count savings do not become wall-clock savings. The premise of this work.
16	The Sparse Frontier: Sparse Attention Trade-offs in Transformer LLMs (Nawrot et al.)	arXiv preprint (2504.17768); ACL Findings 2026 reported, not shown in the paper	2026	Estimation cost limits sparse prefill. Reports FLOPs, not wall-clock time, and names as unknown how close a recall-maximising mask is to a task-optimal one, which is the G3 question. No host-device model.

#	Paper	Venue	Year	Relation to this work
17	MIInference 1.0: Accelerating Pre-filling for Long-Context LLMs via Dynamic Sparse Attention (Jiang et al.)	NeurIPS	2024	Source of the mean-pool and vertical-slash estimators; reports 5% to 30% of time spent building indices on one A100.
18	XAttention: Block Sparse Attention with Antidiagonal Scoring (Xu et al.)	ICML	2025	Calibrated-threshold estimator used as a baseline.
19	DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads (Xiao et al.)	ICLR	2025	Head-differentiated attention; used as a baseline for H4.
20	FlexPrefill: A Context-Aware Sparse Attention Mechanism for Efficient Long-Sequence Inference (Lai et al.)	ICLR	2025	Adapts sparsity per head and prompt; shows adaptive estimators are not new.
21	Twilight: Adaptive Attention Sparsity with Hierarchical Top-p Pruning (Lin et al.)	NeurIPS	2025	Adaptive sparsity budgets; related estimator design.
22	Improving the Performance of LLM Inference: Understanding the Role of CUDA Graphs (Franco et al.)	IEEE LANMAN	2026	Measures how CUDA Graphs remove launch overhead. Relevant to our device-cost measurement.
23	I/O-Aware PIM Acceleration for Long-Sequence LLM Inference with Hybrid Sparse Attention, PILOT (Lu et al.)	IEEE IPDPS	2026	Hardware for sparse prefill and decode. Orthogonal: new hardware, not host-GPU modelling.
24	iMLBench: A Machine Learning Benchmark Suite for CPU-GPU Integrated Architectures (Zhang et al.)	IEEE Transactions on Parallel and Distributed Systems (TPDS)	2021	CPU-GPU co-execution on integrated chips; our pairings are discrete PCIe systems.
25	Producing Wrong Data Without Doing Anything Obviously Wrong! (Mytkowicz et al.)	ASPLOS	2009	Measurement bias in systems evaluation; motivates our pre-registered protocol.
26	How Much Dense Attention is Necessary? Oracle-Guided Sparse Prefill for Full/GQA Layers in Hybrid Long-Context Models (Wang et al.)	arXiv preprint (2606.07703)	2026	Uses an attention-mass oracle as a reference and states it is not a global optimum. The per-query oracle stays within 1 point of dense on Qwen models, but support shared by 64 queries collapses at small budgets. Oracle masks are a premise, not our contribution. Data and code not released.
27	AB-Sparse: Sparse Attention with Adaptive Block Size for Accurate and Efficient Long-Context Inference	arXiv preprint (2605.12110)	2026	Decoding, not prefill. Recall against block size (16 to 64) differs by head and is stable across inputs; accuracy improves with recall at a fixed budget, but accuracy at the same recall is not tested. Block size is therefore a control in our study.
28	Token Sparse Attention: Efficient Long-Context Inference with Interleaved Token Selection	ICML	2026	Reports prefill latency on two GPUs (A100, A6000); an example of single-paper, two-machine evidence.

#	Paper	Venue	Year	Relation to this work
29	APEX: An Extensible and Dynamism-Aware Simulator for Automated Parallel Execution in LLM Serving (Lin et al.)	arXiv preprint (2411.17651)	2025	Serving simulator built from per-device operation profiles. Predicts speedups between parallel plans (10.7% average error), but its absolute per-token time is consistently below measured, which the authors attribute to leaving out operations other than the key ones. No host-side term. Supports G1; not used as a baseline.

Also read in full and judged distant (supplied by the researcher): Kohli et al., IEEE HPEC 2025 (CPU, GPU and NPU benchmarking on one integrated edge chip); HyGIN, IEEE Computer Architecture Letters 2026 (CPU- versus GPU-initiated communication for multi-GPU training); Elango et al., IEEE INFOCOM 2026 (CPU-GPU frequency interaction in a 5G stack, used above as a confounder reference); PLA, IEEE TCAD 2026 (an accelerator with token pruning for point-cloud LLMs; its finding that attention concentrates with depth is why recall is also reported by layer depth); LLMedger, IEEE ICFEC 2026 (pipeline and tensor parallelism scheduling for an encoder-decoder model on CPU-only edge nodes).

Withdrawn. SparkV (arXiv 2604.21231), listed in earlier drafts, has been withdrawn by its authors and is no longer cited.

How each entry was checked. Entries 1, 2, 3, 4, 7, 8, 9, 10, 11, 12, 16, 26, 27 and 29 were read in full. Entry 29, a serving simulator also called APEX, is a different paper from entry 4. The others are described from abstracts and search summaries and will be checked against the full papers before the related-work section is written. Venues and years were checked by web search on 4 October 2026. Preprints (marked as such) whose peer-reviewed venue, if any, should be re-checked before submission. A search of IEEE Xplore and Scopus will follow before the related-work section is written.